Lập trình mạng · Chương 2

Bài toán ứng dụng: web server tối giản trên TCP

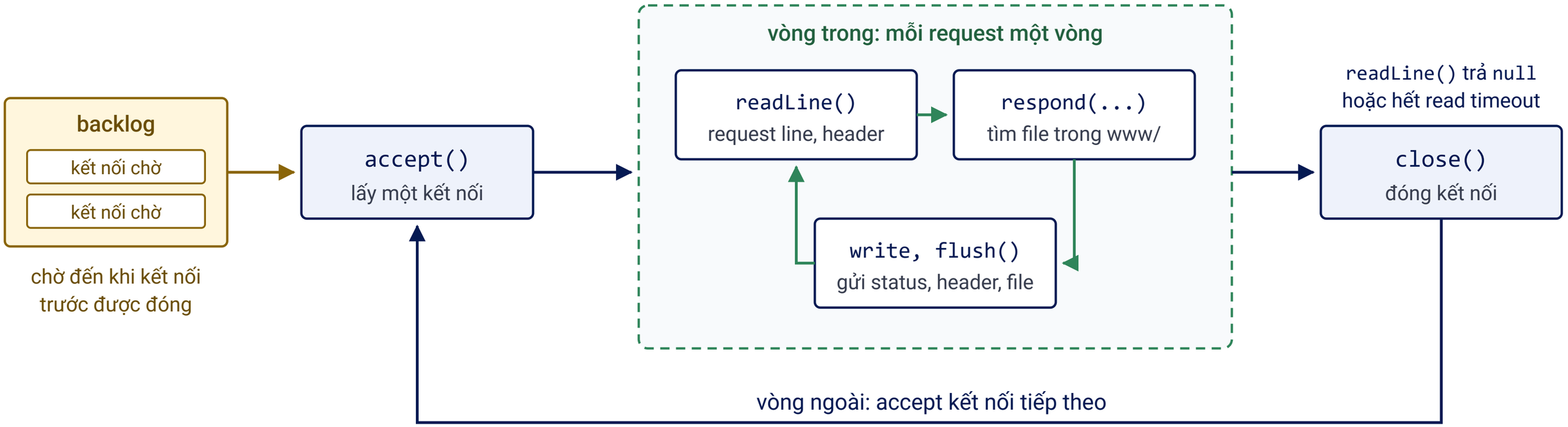
TS. Đặng Ngọc Hùng

Web server tối giản ghép HTTP, kết nối và timeout vào một server thật

1. Đã có: request, response của **HTTP** (Chương 1), kết nối TCP, read timeout, half-close
2. Cần: trả **file** theo đường dẫn với mã trạng thái đúng, phục vụ **nhiều client** lần lượt
3. Và cần **kiểm thử tự động**: chạy lại các trường hợp thử sau mỗi lần sửa mã

> **Câu hỏi của phần:** ghép các cơ chế đã học thành một web server tối giản như thế nào, và kiểm thử server đó ra sao?

Server tuần tự phục vụ xong một kết nối rồi mới accept kết nối sau



Server tuần tự: vòng ngoài accept từng kết nối, vòng trong trả lời từng request; kết nối sau chờ trong backlog.

Mọi kết nối cùng local port 9199, phân biệt bằng 4-tuple

listening socket
`port 9199`
không trao đổi dữ liệu

kết nối	địa chỉ client	port client	địa chỉ server	port server
#1	127.0.0.1	52645	127.0.0.1	9199
#2	127.0.0.1	52646	127.0.0.1	9199
#3	127.0.0.1	52648	127.0.0.1	9199

khác nhau: ephemeral port
hệ điều hành phía client cấp

ba cột còn lại
giống nhau

chỉ cần một trong bốn giá trị khác nhau là **hai kết nối khác nhau**

4-tuple (địa chỉ và port của hai đầu) định danh một kết nối TCP. Server không cấp port mới cho mỗi client.

Vòng lặp ngoài: một kết nối tại một thời điểm, lỗi của một kết nối không dừng server

Java

```
void serve() {
    while (!listener.isClosed()) {
        try (Socket connected = listener.accept()) {  // một kết nối tại một thời điểm
            connected.setSoTimeout(5_000);  // read timeout: client không gửi gì quá 5 giây
            log.println("accepted " + fourTuple(connected));
            int requests = handle(connected);  // phục vụ đến khi client đóng kết nối
            log.println("closed, requests: " + requests);
        } catch (IOException e) {  // gồm cả SocketTimeoutException: lỗi của một kết nối không dừng server
            log.println("connection error: " + e.getMessage());
        }
    }
```

Kết quả chạy

```
listening on port 9199, serving www/
accepted local 127.0.0.1:9199, remote 127.0.0.1:52645
  GET / HTTP/1.1 -> 200 OK
closed, requests: 1
accepted local 127.0.0.1:9199, remote 127.0.0.1:52646
  GET /nothing.html HTTP/1.1 -> 404 Not Found
closed, requests: 1
accepted local 127.0.0.1:9199, remote 127.0.0.1:52648
  GET /hello.txt HTTP/1.1 -> 200 OK
closed, requests: 1
```

Vòng lặp trong: đọc request, tìm file, gửi status line, header và body

Java

```
String[] parts = requestLine.split(" ");  // phương thức, đường dẫn, phiên bản
String path = parts[1].equals("/") ? "/index.html" : parts[1];
Path file = root.resolve(path.substring(1));  // đường dẫn trong thư mục www/
if (!Files.isRegularFile(file)) {
    return send(out, "404 Not Found", "text/plain; charset=utf-8", "404 Not Found\n".getBytes(UTF_8));
}
return send(out, "200 OK", contentType(path), Files.readAllBytes(file));
static String send(OutputStream out, String status, String type, byte[] body) throws IOException {
    String head = "HTTP/1.1 " + status + "\r\n"
            + "Content-Type: " + type + "\r\n"
            + "Content-Length: " + body.length + "\r\n"  // client biết body dài bao nhiêu byte
            + "\r\n";
```

Kết quả chạy

```
$ curl -s -o /dev/null -w "%{http_code} %{content_type}\n" http://127.0.0.1:9199/
200 text/html; charset=utf-8
$ curl -s -o /dev/null -w "%{http_code} %{content_type}\n" http://127.0.0.1:9199/nothing.html
404 text/plain; charset=utf-8
$ curl -i http://127.0.0.1:9199/hello.txt
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 6

hello
```

Integration test chạy server thật ở port 0 và kiểm từng mã trạng thái

Java

```java
@BeforeAll
static void startServer() throws IOException {
    listener = new ServerSocket(0, 50, InetAddress.getLoopbackAddress());  // port 0: hệ điều hành cấp port trống
    Thread server = new Thread(() -> new WebServer(listener, Path.of("www"), NO_LOG).serve());
    server.setDaemon(true);
    server.start();  // server chạy song song với test; thread học ở nhóm đa luồng
}

@Test
void missingFile() throws IOException {
    assertEquals("HTTP/1.1 404 Not Found", statusLine("GET /nothing.html HTTP/1.1"));
}
```

Kết quả chạy

```
└─ JUnit Jupiter ✓
   └─ WebServerTest ✓
      ├─ portZeroGetsEphemeralPort() ✓
      ├─ existingFile() ✓
      ├─ rootServesIndex() ✓
      ├─ missingFile() ✓
      ├─ pathOutsideRoot() ✓
      └─ otherMethod() ✓
Test run finished after 153 ms
```

Một client không gửi gì làm mọi client sau phải chờ

Java

```
connected.setSoTimeout(5_000);  // read timeout: client không gửi gì quá 5 giây
int requests = handle(connected);  // phục vụ đến khi client đóng kết nối
```

Kết quả chạy

```
listening on port 9199, serving www/
accepted local 127.0.0.1:9199, remote 127.0.0.1:52650
connection error: Read timed out
accepted local 127.0.0.1:9199, remote 127.0.0.1:52652
  GET /hello.txt HTTP/1.1 -> 200 OK
closed, requests: 1
```

- curl đến sau **1 giây**: nhận response sau **4,06 giây**, thay vì vài mili giây
- Server tuần tự bị chặn ở `readLine()` của kết nối đầu; kết nối sau đã bắt tay xong, **nằm chờ trong backlog**

Câu hỏi 1

Ba client trên cùng một máy kết nối lần lượt tới web server ở port 9199. Ba kết nối được phân biệt bằng gì?

A. Bằng 4-tuple: địa chỉ và port của hai đầu; ở đây ba kết nối khác nhau ở port client.

B. Server cấp cho mỗi kết nối một port mới sau khi accept.

C. Bằng thứ tự gọi accept: kết nối thứ nhất, thứ hai, thứ ba.

D. Bằng địa chỉ IP của client.

Câu hỏi 1

Ba client trên cùng một máy kết nối lần lượt tới web server ở port 9199. Ba kết nối được phân biệt bằng gì?

A. Bằng 4-tuple: địa chỉ và port của hai đầu; ở đây ba kết nối khác nhau ở port client.

B. Server cấp cho mỗi kết nối một port mới sau khi accept.

C. Bằng thứ tự gọi accept: kết nối thứ nhất, thứ hai, thứ ba.

D. Bằng địa chỉ IP của client.

Một kết nối TCP được định danh bằng 4-tuple. Cả ba kết nối có local port 9199 và cùng địa chỉ 127.0.0.1 ở hai đầu; chỉ port client (ephemeral port) khác nhau. Server không cấp port mới cho kết nối.

Câu hỏi 2

Một client kết nối tới web server tuần tự nhưng không gửi request nào. Vì sao các client đến sau phải chờ?

A. Backlog chỉ chứa được một kết nối.

B. Kết nối của các client sau bị từ chối (connection refused) cho đến khi client đầu đóng.

C. TCP chỉ cho phép một kết nối tới mỗi port tại một thời điểm.

D. Server đang bị chặn ở `readLine()` của kết nối đầu, nên chưa gọi `accept()` cho các kết nối sau.

Câu hỏi 2

Một client kết nối tới web server tuần tự nhưng không gửi request nào. Vì sao các client đến sau phải chờ?

A. Backlog chỉ chứa được một kết nối.

B. Kết nối của các client sau bị từ chối (connection refused) cho đến khi client đầu đóng.

C. TCP chỉ cho phép một kết nối tới mỗi port tại một thời điểm.

D. Server đang bị chặn ở `readLine()` của kết nối đầu, nên chưa gọi `accept()` cho các kết nối sau.

Kết nối của client sau đã bắt tay xong và nằm trong backlog; chỉ có server chưa gọi `accept()` vì còn chờ ở `readLine()` của kết nối đầu, đến khi read timeout hết hạn. TCP cho phép nhiều kết nối tới cùng một port.

Tổng kết

1. **Server tuần tự**: vòng ngoài accept từng kết nối, vòng trong trả lời từng request đến khi `readLine()` trả `null`

2. Mọi kết nối cùng port 9199, phân biệt bằng **4-tuple**; port client là **ephemeral port**

3. **Integration test** chạy server thật ở **port 0**, kiểm từng mã trạng thái

Từ khóa cần nhớ server tuần tự · 4-tuple · integration test · port 0 · ephemeral port

Mở rộng

Chủ đề nâng cao

- Giới hạn độ dài request line và số header
- Integration test cho lỗi mạng: client đóng giữa chừng, kết nối bị reset

Bài tập mở rộng

Thêm phương thức HEAD (status line và header như GET, không có body); viết integration test cho phương thức mới.

Tiếp theo

Đa luồng: thread và race condition